

Abstract of the PhD Dissertation:
Deep Inference for Graphical Theorem Proving

Pablo Donato

1 Introduction

Proof assistants are software tools used to rigorously verify mathematical reasoning. Contemporary systems such as *Coq* [27], *Lean* [20], and *Isabelle* [22] offer powerful frameworks for constructing formal proofs, yet they remain notoriously difficult to learn and use. This usability barrier affects not only newcomers but also experienced users working in complex proof settings, limiting the application potential of proof assistants in two respects:

1. their use beyond the sole purpose of verifying correctness of proofs, for instance as expressive programming languages, or as a medium for understanding and exploring new mathematical concepts interactively;
2. hence their adoption outside of academic research in industry and education.

In this dissertation, we are concerned primarily with the *human*-centered aspects (or “front-end”) of proof assistants: we aim to provide smoother means for the user to communicate her intent to the proof assistant, and conversely for the proof assistant to communicate its results and suggestions on how to solve problems. Furthermore, we restrict the scope of our investigations to the purely *logical* aspects at the heart of proof assistants — that is, the exact representation and manipulation of logical statements for the purpose of deductive reasoning.

The core thesis is that these challenges are not inherent to formal logic itself, but stem in great part from the *textual* mode of representation and interaction imposed by current tools, from the imperative (tactic-based) languages of Coq and Lean — or their more declarative counterparts in Isabelle — to the foundational symbolic formalisms of proof theory and type theory that those languages build upon.

To address this, we introduce a novel paradigm called *Proof-by-Action* (PbA), in which users construct proofs through familiar graphical actions — such as pointing, clicking and dragging — performed directly on the logical content. Inspired by the *direct manipulation* principle from human-computer interaction [25], as well as earlier techniques like *Proof-by-Pointing* (PbP) [4] and *Proof-by-Linking* (PbL) [8], PbA seeks to bridge the gap between the user’s intent and the underlying formal system. Each user interaction is interpreted as a meaningful proof step, and the challenge is to ensure that these actions are semantically grounded, precise, and general enough to express a wide range of logical reasoning in a concise way.

To provide this foundation, we draw on the framework of *deep inference*, a modern approach to proof theory that enables fine-grained, local transformations within proofs. Deep inference was first introduced under this name by Guglielmi in the context of his *calculus of structures* [17], and it offers the flexibility and structural expressiveness needed to model direct manipulation in a formally sound way.

Chapter 1 of the dissertation introduces informally the necessary conceptual and historical background from the *proof theory* and *interactive theorem proving* literature, before presenting the motivations, research goals and methodology summarized above. It then goes on to outline the structure and main contributions of the thesis, as we are about to do in some more details in the next sections. To give a brief overview, the thesis is divided into two parts:

The first part (Chapters 2–6) formalizes PbA in the traditional setting of *symbolic* logic. It introduces *Actema* [12], a graphical user interface (GUI) where gestural actions perform logical inferences, and defines a semantic framework specifically for drag-and-drop actions based on the *subformula linking* technique in deep inference [6]. It also explores more advanced features like inductive reasoning and simplification, extends PbA to classical logic via multi-conclusion sequents, and integrates the interface into Coq via the `coq-actema` plugin [11], translating GUI actions into verified proof terms.

The second part (Chapters 7–10) investigates *iconic* systems for logic, where goals are represented as diagrams rather than formulas. It introduces *bubble calculi*, which extend the nested sequents of Brünnler [5] with topological structure, and revisits C. S. Peirce’s *existential graphs* [23] in an intuitionistic light through the design of the *flower calculus* [9]. The latter system is the foundation for the *Flower Prover*, another prototype of GUI for interactive proof building designed to work well with modern devices based on touch interactions.

Publications Most of the content of Chapters 2 and 3 has been previously published in [10]. A shortened version of Chapters 9 and 10 has been published in [9]. All other chapters present completely original and personal work.

In sum, the thesis contributes both a theoretical framework and a set of practical tools for reimagining how users interact with formal proofs. By treating interaction as a first-class element of proof theory, it opens the way for new kinds of logical literacy that are more intuitive, visual, and expressive.

2 Proof-by-Action

Chapter 2 explains how both *click* and *drag-and-drop* (DnD) actions upon the formulas of a sequent can implement proof construction operations corresponding to the core logic; that is, how they deal with logical connectives, quantifiers, and equality. We present these core principles through various illustrations and examples in first-order logic. More advanced features of the Proof-by-Action paradigm that go beyond the core logic are illustrated in Chapter 4, and Chapter 6 explains how it can be integrated in a mainstream proof assistant.

This chapter focuses mostly on two broad categories of DnD actions: *forward steps*, where two existing facts are combined to deduce a new one, and *backward steps*, where a known fact is used to transform the goal. These operations are backed by a proof-theoretic technique known as *subformula linking* that will be formally defined in the next chapter. It can roughly be understood as a kind of operational semantics, where the result of a DnD action is computed by applying rewriting rules inside logical formulas. This is illustrated in examples by giving both the resulting new statement and its derivation computed by the system. The set of rewriting rules is also given at the end of the chapter for reference.

We have started to implement the paradigm in a prototype named *Actema* (for “**A**ctive **m**athematics”) running through a web HTML/JavaScript interface. At the time of writing, a standalone version of the prototype (i.e., which does not use an existing theorem prover as its backend) is publicly available online [12]. This possibility to experiment in practice, even though yet

on a small scale, gave valuable feedback for crafting the way DnD actions are to be translated into proof construction steps in an intuitive and practical way. A description of the overall architecture and implementation design of Actema is provided in Chapter 6.

Overall, this chapter serves as a practical and motivational entry point into the thesis, demonstrating how logical reasoning can be made more intuitive and accessible through carefully designed graphical interactions that remain formally sound.

3 Subformula Linking

In Chapter 3, we engage in a thorough analysis of the logical semantics of DnD actions, which were introduced informally through examples in Chapter 2. We do this mainly from the formal perspective of deep inference proof theory, following the original work of K. Chaudhuri on *subformula linking* [6]. But we always keep in mind the intended application to proof assistants, by motivating various design choices — actual or prospective — as ways to improve the *user experience* (*UX*) of interactive proof building.

3.1 Linkages This section introduces the notion of a *linkage* as the formal representation of a DnD action connecting two selected subformulas. It defines the standard deep inference notions of *context* and *polarity*, and details how linkages are structured to determine the kind of interaction intended by the user.

3.2 Validity Here, a precise criterion is established for when a linkage is considered valid, combining two key conditions: opposite polarities between the linked subterms and their unifiability. These conditions ensure that the interaction leads to productive simplification rather than vacuous transformation. This is a novelty and the main point of divergence with the original approach of Chaudhuri.

3.3 Describing DnD Actions This section outlines the full process by which a DnD action is transformed into a valid proof step, including the generation, validation, and execution of linkages. It defines a deterministic and terminating procedure that guarantees logical soundness by alignment with the system’s rewriting rules.

3.4 Soundness This part formally proves that all rewriting steps preserve logical validity (Theorem 3.4.1). It establishes contextual soundness (Lemma 3.4.4) by ensuring that every transformation corresponds to a provable sequent in intuitionistic first-order logic with equality.

3.5 Productivity The section shows that for every valid linkage, the system can always reach a final interaction point (corresponding to an identity axiom or substitution rule for equality) in a finite number of steps (Theorem 3.5.2). This ensures that DnD actions are not just sound but also effective in progressing toward the completion of proofs.

3.6 Focusing The set of rewriting rules for subformula linking turns out to be non-confluent. To resolve ambiguities in rule application, this section adapts the focusing discipline well known from the sequent calculus literature, preferring invertible rules to guide the rewriting strategy. This helps avoid dead ends and heuristically makes the semantics of DnD actions more natural (at least in common situations).

3.7 Completeness This section demonstrates that the set of DnD actions, taken alone, can form a complete deductive system for first-order intuitionistic logic. It constructs a simulation argument to relate the deep inference system to standard sequent calculi (Theorem 3.7.1).

3.8 Related Works The final section surveys related literature, comparing this approach to subformula linking with alternatives in deep inference [7], proof nets [16, 26, 18], automated theorem proving [1], program verification [21] and functional programming [13]. It highlights how the notion of valid linkage simplifies usability while retaining semantic power.

4 Proof-by-Action in Practice

In the previous chapters, we explained the core principles of our so-called Proof-by-Action paradigm and especially of its drag-and-drop actions, first through basic and abstract examples in Chapter 2, and then from a proof-theoretical perspective in Chapter 3. The goal of this chapter is to provide a better sense of what proofs by action/DnD look like in practice, and how they compare to more traditional approaches to interactive theorem proving. To that effect, we perform a case study of a few select examples, unrolling and commenting in detail one or many of their proofs. Although still basic, they are fully fledged, concrete logical riddles or mathematical problems that one might give as exercises to an undergraduate student learning formal proofs. Note that our analysis stays quite informal and opinionated: a more systematic approach such as a user study would allow for a better evaluation of our paradigm, but at the time of writing of this thesis the Actema prototype was not mature enough to conduct a project of this scale.

4.1 Forward Reasoning This section presents a proof of a simple logical riddle taken from the Edukera project [24] within the Actema interface. The visual approach reduces the cognitive overhead of naming hypotheses and structuring tactic sequences, especially for beginners. A comparison with the equivalent Coq script shows that the graphical version is not only more accessible but often faster to construct.

4.2 Sets and Functions Here, the paradigm is extended to reasoning about functions and sets. We distinguish between nominal, axiomatic, and computational *definitions*, each supported by corresponding contextual menu actions in Actema such as **Simplify** or **Unfold**. The case study demonstrates that PbA can accommodate both explicit and implicit styles of proof found in various proof assistants.

4.3 Peano Arithmetic The final section tackles a more complex proof: the commutativity of addition on natural numbers, using *induction*. It shows how inductive steps can be triggered via contextual actions, and how automation (e.g. simplification and lemma search) can be performed with more power, precision and ease than is possible with a textual language. The close match between the graphical proof and a Coq script suggests promising directions for compiling PbA proofs to traditional languages.

5 Parallel Conclusions and Classical Logic

This chapter extends the Proof-by-Action paradigm to support sequents with multiple conclusions, as used for classical logic by Gentzen [15]. It explores how graphical reasoning can be adapted to this setting by introducing a new form of subformula linking — *parallel interaction* — and formalizes it through an original set of rules and linking semantics [6].

5.1 Backtracking Discusses the limitations of single-conclusion systems in handling disjunctive goals and the cognitive burden of backtracking in traditional proof assistants. The section motivates the need for invertible rules and the expressive power of multi-conclusion sequents for reducing backtracking, as noticed by other authors [2].

5.2 Implementation in Theorem Provers Notes that despite the advantages, most modern proof assistants avoid multi-conclusion sequents due to UI complexity. The Actema interface circumvents these limitations through direct manipulation, making multiple conclusions manageable.

5.3 Click Actions Explains how click actions in Actema are extended to support multi-conclusion sequents. Modifications to disjunction and implication actions allow Actema to capture both intuitionistic and classical behavior, depending on how extra conclusions are treated.

5.4 Drag-and-Drop Actions Introduces the concept of *parallel linking* between two conclusions, interpreted as disjunction. This gives rise to a third mode of interaction — parallel reasoning — complementing forward and backward reasoning, and supported by a new logical operator.

5.5 Metatheory of Parallel Reasoning Establishes the soundness of parallel rules in the classical setting, while noting their incompatibility with intuitionistic logic. Updates are made to the validity conditions for linkages to allow for parallel interactions, and extensions to the core metatheory of Chapter 3 are proposed to accommodate classical completeness and productivity.

6 Integration in a Proof Assistant

This chapter describes how the Proof-by-Action paradigm has been integrated into the Coq proof assistant through the `coq-actema` system [11]. The goal is to connect the graphical interface developed in earlier chapters with an industrial-strength proof assistant, enabling users to benefit from its large ecosystem of existing tools and formalizations. This is also a necessary step to test how the PbA paradigm scales to real-world mathematical developments.

6.1 Actema Introduces the architecture of the Actema web application [12], which separates a JavaScript frontend from an OCaml backend compiled via `js_of_ocaml` [28]. The two components communicate via an object-oriented API, enabling interaction between the UI and the proof engine.

6.2 Why a Plugin? Argues for integrating Actema via a dedicated Coq plugin rather than modifying Coq itself or relying on high-level protocols like SerAPI [14]. This design enables Actema to function as an alternative, graphical proof view within standard Coq IDEs such as CoqIDE or Proof General.

6.3 The `coq-actema` System Describes the full-stack communication setup, where a plugin running inside Coq interacts with the Actema frontend over HTTP. This enables real-time, bidirectional proof interaction and allows packaging the interface as a standalone Electron application for ease of deployment.

6.4 Interaction Protocol Details the communication protocol using UML-style sequence diagrams. These diagrams illustrate how proof goals are exported, actions are queried and selected, and proof steps are sent back to Coq to be compiled as executable tactics, capturing subformula linking and other PbA operations.

6.5 Compiling Actions Explains how user actions in the graphical interface are serialized via ATD [19] and compiled into Coq tactics through metaprogramming, using $\mathcal{L}_{tac}$ to implement the semantics of subformula linking in a deep embedding of first-order logic.

6.6 Future Work Outlines limitations of the current system, such as incomplete support for advanced Coq features and lack of robust proof maintenance tools. It also addresses the issue of proof evolution and proposes directions for improving the long-term usability of Actema for real-world developments.

7 Asymmetric Bubble Calculus

This chapter introduces the *asymmetric bubble calculus* as a new kind of nested sequent system [5]. By internalizing the notion of subgoals as “bubbles”, it provides a more topological and hierarchical way to represent logical contexts. This enables efficient sharing of hypotheses between subgoals, as well as more local proof steps.

7.1 The chemical metaphor Explains how Actema’s interface can be interpreted through a *chemical* metaphor, with formulas viewed as reactants that can break or merge under user-driven proof actions, inspired by the *chemical abstract machine* (CHAM) of Berry and Boudol [3]. This perspective guides the design of *bubble* boundaries akin to the CHAM’s *membrane* mechanism, that manage context scope without relying on multiple tabs or separate subgoals.

7.2 Bubbles and solutions Introduces bubbles as a device for delineating subgoals and allowing selective flow of hypotheses. The idea of a “*solution*” generalizes a sequent by blending hypotheses, conclusions and subgoals into a single space, giving users a more direct handle on scoping.

7.3 Asymmetric calculus Presents the collection of local rewriting rules constituting the asymmetric bubble calculus, specialized for single-conclusion intuitionistic logic. Each rule concerns either structural manipulations (like context sharing) or logical connectives (like introductions and eliminations), and it is shown how these rules remain faithful to a standard sequent system.

7.4 Back to Proof-by-Action Connects the rules for manipulating bubbles to the gestural actions of PbA described in the first part of the thesis. The membranes of bubbles can be represented in a GUI, suggesting a continuous, topological model of scoping and subgoal management providing a more physical way to interact with the proof state.

8 Symmetric Bubble Calculi

9 Existential Graphs

10 Flower Calculus

References

- [1] Andrews. “Refutations by Matings”. In: *IEEE Transactions on Computers* C-25.8 (1976), pp. 801–807.
- [2] Edward W. Ayers. “A Tool for Producing Verified, Explainable Proofs.” en. PhD thesis. University of Cambridge, 2021.
- [3] Gerard Berry and Gerard Boudol. “The chemical abstract machine”. In: *Proceedings of the 17th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. POPL ’90. New York, NY, USA: Association for Computing Machinery, Dec. 1989, pp. 81–94. ISBN: 978-0-89791-343-0. DOI: 10.1145/96709.96717. URL: <https://doi.org/10.1145/96709.96717> (visited on 05/02/2022).

- [4] Yves Bertot, Gilles Kahn, and Laurent Théry. “Proof by pointing”. In: *Theoretical Aspects of Computer Software*. Ed. by Masami Hagiya and John C. Mitchell. Red. by Gerhard Goos and Juris Hartmanis. Vol. 789. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 1994, pp. 141–160. ISBN: 978-3-540-57887-1 978-3-540-48383-0. DOI: 10.1007/3-540-57887-0_94. URL: http://link.springer.com/10.1007/3-540-57887-0_94 (visited on 04/21/2020).
- [5] Kai Brännler. “Deep sequent systems for modal logic”. en. In: *Archive for Mathematical Logic* 48.6 (July 2009), pp. 551–577. ISSN: 0933-5846, 1432-0665. DOI: 10.1007/s00153-009-0137-3. URL: <http://link.springer.com/10.1007/s00153-009-0137-3> (visited on 01/10/2024).
- [6] Kaustuv Chaudhuri. “Subformula Linking as an Interaction Method”. In: *Interactive Theorem Proving*. Ed. by Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie. Red. by David Hutchison et al. Vol. 7998. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 386–401. ISBN: 978-3-642-39633-5 978-3-642-39634-2. DOI: 10.1007/978-3-642-39634-2_28. URL: http://link.springer.com/10.1007/978-3-642-39634-2_28 (visited on 04/21/2020).
- [7] Kaustuv Chaudhuri. “Subformula Linking for Intuitionistic Logic with Application to Type Theory”. In: *Automated Deduction - CADE 28 - 28th International Conference on Automated Deduction, Virtual Event, July 12-15, 2021, Proceedings*. Ed. by André Platzer and Geoff Sutcliffe. Vol. 12699. Lecture Notes in Computer Science. Springer, 2021, pp. 200–216. DOI: 10.1007/978-3-030-79876-5_12. URL: https://doi.org/10.1007/978-3-030-79876-5_12.
- [8] Kaustuv Chaudhuri et al. *Certifying Proof-By-Linking*. en. Sept. 2022. URL: <https://inria.hal.science/hal-04317972> (visited on 12/09/2023).
- [9] Pablo Donato. “The Flower Calculus”. In: *9th International Conference on Formal Structures for Computation and Deduction (FSCD 2024)*. Ed. by Jakob Rehof. Vol. 299. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 5:1–5:24. ISBN: 978-3-95977-323-2. DOI: 10.4230/LIPIcs.FSCD.2024.5. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2024.5>.
- [10] Pablo Donato, Pierre-Yves Strub, and Benjamin Werner. “A drag-and-drop proof tactic”. In: *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2022. Philadelphia, PA, USA: Association for Computing Machinery, 2022, pp. 197–209. ISBN: 9781450391825. DOI: 10.1145/3497775.3503692. URL: <https://doi.org/10.1145/3497775.3503692>.
- [11] Pablo Donato and Benjamin Werner. *coq-actema*. 2023. URL: <https://github.com/Champitoad/coq-actema> (visited on 02/24/2024).
- [12] Pablo Donato et al. *The Actema prototype, standalone version*. <https://actema.xyz>. 2021.
- [13] Conal M. Elliott. “Tangible Functional Programming”. In: *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming*. ICFP ’07. Freiburg, Germany: Association for Computing Machinery, 2007, pp. 59–70. ISBN: 9781595938152. DOI: 10.1145/1291151.1291163. URL: <https://doi.org/10.1145/1291151.1291163>.
- [14] Emilio Jesús Gallego Arias. “SerAPI: Machine-Friendly, Data-Centric Serialization for COQ”. working paper or preprint. Oct. 2016. URL: <https://hal-mines-paristech.archives-ouvertes.fr/hal-01384408>.

- [15] Gerhard Gentzen. “Untersuchungen über das logische Schließen. I”. de. In: *Mathematische Zeitschrift* 39.1 (Dec. 1935), pp. 176–210. ISSN: 1432-1823. DOI: 10.1007/BF01201353. URL: <https://doi.org/10.1007/BF01201353> (visited on 12/10/2023).
- [16] Jean-Yves Girard. “Linear logic”. In: *Theoretical Computer Science* 50.1 (1987), pp. 1–101. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(87\)90045-4](https://doi.org/10.1016/0304-3975(87)90045-4). URL: <http://www.sciencedirect.com/science/article/pii/0304397587900454>.
- [17] Alessio Guglielmi. *A Calculus of Order and Interaction*. Tech. rep. Technische Universität Dresden, 1999. URL: https://www.researchgate.net/publication/2807151_A_Calculus_of_Order_and_Interaction.
- [18] Dominic J. D. Hughes. *First-order proofs without syntax*. 2019. arXiv: 1906.11236 [math.LO]. URL: <https://arxiv.org/abs/1906.11236>.
- [19] Martin Jambon. *OCaml support – atdgen*. <https://atd.readthedocs.io/en/latest/atdgen.html>. 2023. (Visited on 07/05/2023).
- [20] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *Automated Deduction – CADE 28*. Ed. by André Platzer and Geoff Sutcliffe. Cham: Springer International Publishing, 2021, pp. 625–635. ISBN: 978-3-030-79876-5.
- [21] Ike Mulder and Robbert Krebbers. “Unification for Subformula Linking under Quantifiers”. In: *Proceedings of the 13th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2024. New York, NY, USA: Association for Computing Machinery, Jan. 2024, pp. 75–88. ISBN: 9798400704888. DOI: 10.1145/3636501.3636950. URL: <https://dl.acm.org/doi/10.1145/3636501.3636950> (visited on 01/21/2024).
- [22] Tobias Nipkow, Lawrence C Paulson, and Markus Wenzel. *Isabelle/HOL: a proof assistant for higher-order logic*. Vol. 2283. Springer Science & Business Media, 2002.
- [23] Don D. Roberts. *The Existential Graphs of Charles S. Peirce*. Berlin, Boston: De Gruyter Mouton, 1973. ISBN: 9783110226225. DOI: doi:10.1515/9783110226225. URL: <https://doi.org/10.1515/9783110226225>.
- [24] Benoit Rognier and Guillaume Duhamel. “Présentation de la plateforme edukera”. In: *Vingt-septièmes Journées Francophones des Langages Applicatifs (JFLA 2016)*. 2016.
- [25] Shneiderman. “Direct Manipulation: A Step Beyond Programming Languages”. en. In: *Computer* 16.8 (Aug. 1983), pp. 57–69. ISSN: 0018-9162. DOI: 10.1109/MC.1983.1654471. URL: <http://ieeexplore.ieee.org/document/1654471/> (visited on 01/08/2024).
- [26] Lutz Straßburger. “The problem of proof identity, and why computer scientists should care about Hilbert’s 24th problem”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 377.2140 (Mar. 11, 2019), p. 20180038. ISSN: 1364-503X, 1471-2962. DOI: 10.1098/rsta.2018.0038. URL: <https://royalsocietypublishing.org/doi/10.1098/rsta.2018.0038> (visited on 08/08/2020).
- [27] The Coq Development Team. *The Coq Proof Assistant*. Version 8.16. 2022. DOI: 10.5281/zenodo.7313584. URL: <https://doi.org/10.5281/zenodo.7313584>.
- [28] Jérôme Vouillon and Vincent Balat. “From bytecode to JavaScript: the Js_of_ocaml compiler”. en. In: *Software: Practice and Experience* 44.8 (Aug. 2014), pp. 951–972. ISSN: 00380644. DOI: 10.1002/spe.2187. URL: <https://onlinelibrary.wiley.com/doi/10.1002/spe.2187> (visited on 01/06/2023).